

Contents

Implementation Plan: Mixed-Flux + Heterogeneous (Bi-material) Riemann Solver	1
Overview	1
Constraints	1
Math	3
Phase 1: Bi-material central-flux primitive	3
Phase 2: Precompute + store per-face central matrices on the matrix operator	5
Phase 3: Dispatch + lift the mutual-exclusion abort	6
Phase 4: CFL de-rating on the matrix path	8
Phase 5: Driver + config wiring (lift ALL series guards; wire seam_continuous)	9
Phase 6: Unit tests, homogeneous-equivalence gate, Makefile	11
Phase 7 (FINAL): TPV31 heterogeneous end-to-end validation on Frontera	12
Testing Strategy	15
Risk Assessment	16
Tricky existing-code areas this plan touches	16

Implementation Plan: Mixed-Flux + Heterogeneous (Bi-material) Riemann Solver

Worktree: .claude/worktrees/mixed-flux-hetero-riemann (branch feat/mixed-flux-hetero-riemann). *Companion:* EXPLORE_mixed_flux_hetero_riemann.md (exploration + math). *Review:* PLAN_mixed_flux_hetero_riemann_check.md (Rev 5 — 26 defects tracked; Rev-5 verdict: **implementable, no CRITICAL/HIGH open**). **This is Rev 6 of the plan. Rev 2→5 fixed BUG-1...BUG-24 and confirmed an exhaustive 3-guard inventory (G1 spatial_friction.cpp:1162, G2 driver:809-816, G3 bimaterial_wave_operator.inl:569). Rev 6 applies the two non-blocking Rev-5 clean-ups: BUG-25 (the Test 5.3c parser-regression corpus — realize “byte-identical” as a field-by-field extension of tests/unit/test_tpv_config_parse.cpp, add tpv31 configs, DROP BP5 which uses the native seas_bp5_full driver and never calls this parser) and BUG-26 (state that TPV31 is law=“slip_weakening”, so the RK path is AdvanceRKCoupledLSW_Spatial; the central dispatch is law-agnostic in Mult).** No CRITICAL/HIGH findings remain. *Reference:* drdg3d src/mod_wave.F90::get_flux (Zhang 2023). Path-relative roots below are under miniapps/seas/.

File-naming note: the skill template names the output PLAN.md, but the repo root already holds a large unrelated PLAN.md. This plan is written to a dedicated file to avoid clobbering it. The implementer should follow *this* file.

Overview

Today the SEAS-MFEM spatial driver can use **mixed flux** (central flux on fault-adjacent faces) **or** the **heterogeneous bi-material Riemann solver** (interior_flux=“matrix”), but not both: BimaterialWaveOperator::SetMixedFluxMode aborts on any non-None mode, and the driver itself rejects matrix + RK. drdg3d uses the two together (one per-face if: central on fluxtype==1 faces, bi-material Godunov elsewhere). This plan adds a **bi-material central flux** $F^* = \frac{1}{2}(A_{\text{self}} \cdot Q_{\text{self}} + A_{\text{nbr}} \cdot Q_{\text{nbr}})$, dispatches it per-face on the matrix operator, applies the correct CFL de-rating, lifts BOTH abort sites, and tests the combination — gated by the requirement that the homogeneous limit reproduces the existing scalar mixed-flux path byte-for-byte.

The **final deliverable (Phase 7)** is the end-to-end validation target: **SCEC TPV31** (tpv31/), a depth-dependent (vertically-layered bi-material) dynamic-rupture benchmark. TPV31 is the *only* benchmark that needs interior_flux=“matrix” AND benefits from central flux on the fault-adjacent faces, so it is the first job to exercise the merged matrix-bimaterial + RK + central-flux path. Per project policy the TPV31 production run is **Frontera-only**: Phases 1–6 are verified locally on tiny fixtures + unit tests; Phase 7 produces the Frontera config + sbatch and a real-mesh dispatch gate, and the user runs it on Frontera after explicit approval.

Constraints

- **Byte-exact regression contract (hard).** TPV102 / TPV104 / TPV205 / BP5 must remain byte-identical. They run the scalar WaveOperator and/or mixed_flux=none; nothing in this plan may touch those code paths’ numerics. The interior_flux=“matrix” + mixed_flux=none path must also stay byte-exact (test_bimaterial_wave_operator_parity, 22/22). **The existing TPV31 ADER + mixed_flux=none jobs (jobs/tpv31_spatial/tpv31_p2_ader03_*) must also stay byte-exact** — Phase 7 adds a *new* RK +

adjacent job; it does not alter the ADER path. **(BUG-24) The new seam_continuous guard cannot regress that ADER job: with mixed_flux="none", BuildCentralFluxFaceSet_ returns an empty set (wave_operator.inl:1653-1654) and mf_on_=false, so the Phase-2 BuildPerFaceCentralFluxMatrices_ (which walks only central_flux_face_set_) is never entered and the guard never fires — and G1 already allows matrix + mixed_flux=none. The guard fires ONLY for Mode::Coefficient + a central-set shared face with mf_on_ true.**

- **Homogeneous-limit equivalence (hard correctness gate).** With homogeneous material, BimaterialWaveOperator + mixed_flux=adjacent MUST reproduce WaveOperator (scalar) + mixed_flux=adjacent to $\leq 1e-9$ (LU-rounding tolerance, same as R.1.T-1 / parity tests; test_bimaterial_wave_operator_parity.cpp:254-255, 288 uses k_tol = 1e-9).
- **Single-valuedness / conservation (hard correctness gate — BUG-3).** On any fault-adjacent (“central-set”) interior face, the deposited central flux MUST be identical on both sides: $F_{h_e1} == F_{h_e2}$ componentwise, **including under heterogeneity** ($A_{e1} != A_{e2}$). This is the highest-risk invariant; the plan’s storage/dispatch (Phase 2/3) is designed to guarantee it structurally, and Phase 3 Test 3.2 enforces it on a heterogeneous fixture.
- **Interfaces that cannot change:** the base WaveOperator virtual hook signatures InteriorFaceFlux_(int, const real_t*, const real_t*, const real_t*, real_t* F_h_e1, real_t* F_h_e2) (wave_operator.hpp:920-922), SharedInteriorFaceFlux_(int, const real_t*, const real_t*, const real_t*, real_t* F_h) (hpp:927-929), SetMixedFluxMode(MixedFluxMode) (hpp:293), ComputeMaxDt(real_t) (hpp:206); the 9-component (NUM_STATE=9) state layout; BimaterialFlux::ApplyPerFaceFlux signature (godunov_flux_bimaterial.hpp:156-160). InteriorFaceFlux_ fills BOTH F_h_e1 and F_h_e2; SharedInteriorFaceFlux_ fills a single F_h (side 0 only).
- **Reuse, do not fork:** the base BuildCentralFluxFaceSet_ (wave_operator.hpp:898, wave_operator.inl:1650; rank-symmetric, R-1100/R-1206/R-1404 correct) is operator-agnostic and MUST be reused — do not reimplement face selection. central_flux_face_set_ (hpp:894), mf_on_ (hpp:888), mixed_flux_mode_ (hpp:883), cfl_rk_aware_ (hpp:893) are all protected and visible to BimaterialWaveOperator.
- **MPI symmetry:** SetMixedFluxMode must be called with the same mode in the same order on every rank (the base Allgatherv/Allreduce mode cross-check at wave_operator.inl:1593-1610 must stay on the bi-material path — keep Base::SetMixedFluxMode(m) first).
- **Source-edit scope.** SOURCE edits are limited to miniapps/seas/dynamic/ + drivers/spatial_dyn_driver.cpp + spatial/code/spatial_friction.{hpp,cpp} + tests/unit/ (+ tests/parallel/ for the np=2 case) + Makefile. spatial/code/spatial_friction.{hpp,cpp} is a **Rev-5 scope expansion (BUG-22)**: the spatial config struct (MaterialSpec, spatial_friction.hpp:573), its parser, and the config-parse mutual-exclusion guard G1 (spatial_friction.cpp:1162-1167) all live there, so relaxing G1 (BUG-21) and adding [material].seam_continuous (BUG-22) require editing it. It is NOT BP5/friction/solver core (project memory [C2] does not forbid spatial/code/), but every TPV*/TPV31 spatial config parses through it (BP5 uses its own native seas_bp5_full driver and never calls LoadSpatialFrictionConfig, so it is unaffected by this edit) — so any edit MUST be guarded by a parser-regression test (Phase 5 Test 5.3c) proving those spatial configs still parse unchanged. **Phase 7 adds benchmark artifacts only** — tpv31/configs/*.toml and jobs/tpv31_spatial/*.sbatch — which are not source and not under the no-touch tree.
- **Integrator:** central flux is non-dissipative $\Rightarrow$ ADER-O2 is unstable on it (wave_operator.inl \$5.4/\$6.3 note). Mixed flux on the matrix path REQUIRES the RK integrator (cfl_rk_aware_), exactly as on the scalar path. Both the driver guard (Phase 5) and ComputeMaxDt (Phase 4) must enforce this.
- **Material-heterogeneity support boundary (BUG-6 / BUG-10).** The cross-rank neighbour-material exchange is a *local-side stub* (ExchangeBiMaterialNeighbours_, bimaterial_wave_operator.inl:120-190): it stores the LOCAL element’s material as the neighbour’s (inl:162) and only warns (rank 0, inl:165-187) on lateral heterogeneity across a partition seam. The merged central path inherits this: it is correct for Constant and **seam-continuous (depth-only) materials such as TPV31’s depth_profile_1d**, which is the regime the existing parallel TPV31 ADER bimaterial job already runs in. Laterally-varying (Mode::Coefficient varying in-plane) material across a partition seam is **unsupported** until a real cross-rank MPI_Allgatherv exchange is implemented (deferred — documented follow-up). **Enforcement is a declarative config flag material.seam_continuous, NOT a runtime material comparison or probe** (BUG-10/BUG-15/BUG-16): comparing the *stored* neighbour to local is a dead predicate (the stub sets them equal by construction, inl:162); and a material-field *probe* is not viable either — MaterialField has no arbitrary-point evaluator the operator can reach (only EvalAt(elem, ElementTransformation&, IntegrationPoint&), with no transformation for a neighbour point across a rank seam — BUG-15), and an along-face-normal probe conflates legitimate depth variation (TPV31’s depth_profile_1d jumps at depth 2400/5000/10000 m) with the in-plane variation it is meant to catch, false-aborting the deliverable (BUG-16). Instead, Phase 2 **aborts any Mode::Coefficient central shared face unless material.seam_continuous = true is declared** — a deterministic, communication-free gate that makes the

supported regime explicit. TPV31 sets `seam_continuous = true` (it is the same local-side-stub approximation the existing parallel TPV31 ADER job already accepts). See Phase 2 req 4.

- **No local production runs** (project memory): end-to-end validation on the real TPV31 50 m mesh is Frontera-only. Local tests use tiny fixtures ($\leq$ a few hundred elements). The Frontera sbatch is generated, NOT submitted, until the user gives explicit approval.

Math

Bi-material upwind (existing, interior faces): $F_{h_self} = \text{fluxLocal} \cdot Q_{self} + \text{fluxNeighbor} \cdot Q_{nbr}$, where $\text{fluxLocal}/\text{Neighbor}$ come from the `SeisSol` characteristic projection (`godunov_flux_bimaterial.hpp`). Per-side impedances Z^-, Z^+ ; reduces to single-impedance upwind when materials match. Each side applies its OWN Jacobian to the SHARED Riemann state Q^* — this per-side asymmetry is CORRECT for upwind (`bimaterial_wave_operator.inl:262-312`).

Bi-material central (new, fault-adjacent faces): the standard averaged physical flux

$$F^* = \frac{1}{2} (A_{e1} \cdot Q_{e1} + A_{e2} \cdot Q_{e2}) \quad (\text{single-valued across the face})$$

where A_{side} is the **face-normal Jacobian of that side's material in the global frame**, $A_{side} = T \cdot (Ax_{plus_side} + Ax_{minus_side}) \cdot T^{-1}$ (T = face-local $\rightarrow$ global rotation built from `nor`). Per-face matrices $\text{centralE1} = \frac{1}{2} \cdot A_{e1}$ (multiplies Q_{e1}) and $\text{centralE2} = \frac{1}{2} \cdot A_{e2}$ (multiplies Q_{e2}). Properties required by the tests:

- **Homogeneous limit** $A_{e1} == A_{e2} \Rightarrow F^* = \frac{1}{2} \cdot A \cdot (Q_{e1} + Q_{e2}) = \text{GodunovFlux}::\text{Central}$.
- **Single-valued / conservative (BUG-3):** unlike the upwind flux, the central flux has **no shared interface state** — it is the literal average of the two physical fluxes and is the SAME on both sides. The dispatch MUST deposit ONE F^* and copy it to the second side ($F_{h_e2}[c] = F_{h_e1}[c]$), mirroring the scalar reference at `wave_operator.inl:1203-1209` (`flux_.Central(...)` computed once into F_{h_e1} , then $F_{h_e2}[c] = F_{h_e1}[c]$). The per-side operand swap that the upwind build uses (`ResolveFaceFluxOperands_side 1 = self/nbr swapped`) MUST NOT be applied to the central build; doing so yields $F_{h_e2} = \frac{1}{2}(A_{e2} \cdot Q_{e1} + A_{e1} \cdot Q_{e2}) \neq F_{h_e1}$ under heterogeneity — the exact non-conservative pitfall this plan must avoid.
- **Consistency:** $Q_{e1} == Q_{e2} == Q$ with one material $\Rightarrow F^* = A \cdot Q$ (the physical flux).

Do NOT reuse `GodunovFlux::Central` per side under heterogeneity. It computes $\frac{1}{2} \cdot A_{self} \cdot (Q_{self} + Q_{nbr})$ (one Jacobian for both terms — verified at `godunov_flux.cpp:382-423`, line 416 uses `Ax_plus_+Ax_minus_`) $\rightarrow$ a *different, non-single-valued* flux when $A_{e1} \neq A_{e2}$. This is the single most important correctness pitfall (see EXPLORE Gotchas + check BUG-3).

Phase 1: Bi-material central-flux primitive

Goal

`BimaterialFlux` can build the per-face central matrices $\frac{1}{2} \cdot A_{self}$, $\frac{1}{2} \cdot A_{nbr}$ (global frame), verified to reduce to `GodunovFlux::Central` in the homogeneous limit and to use the NEIGHBOUR Jacobian for the neighbour term under heterogeneity.

Files to Modify

- `dynamic/godunov_flux_bimaterial.hpp` — declare the central builder + a derivation comment block (why $\frac{1}{2}A_{self} \cdot Q_{self} + \frac{1}{2}A_{nbr} \cdot Q_{nbr}$ and NOT $\frac{1}{2}A_{self} \cdot (Q_{self} + Q_{nbr})$), mirroring the existing upwind derivation comment.
- `dynamic/godunov_flux_bimaterial.cpp` — implement it, reusing the rotation + scratch-matrix machinery of `BuildPerFaceFluxMatricesGlobal` (`cpp:240-296`).

Files to Create

- `tests/unit/test_bimaterial_central_flux.cpp` — primitive-level tests.

Detailed Requirements

1. Add a static method mirroring `BuildPerFaceFluxMatricesGlobal`:

```

static void BuildPerFaceCentralMatricesGlobal(
    const real_t* nor,
    const GodunovFlux& flux_self,
    const GodunovFlux& flux_nbr,
    mfem::DenseMatrix& centralSelf,    // 9x9:  $\frac{1}{2} \cdot T \cdot (AxPlus\_self + AxMinus\_self) \cdot T^{-1}$ 
    mfem::DenseMatrix& centralNbr);    // 9x9:  $\frac{1}{2} \cdot T \cdot (AxPlus\_nbr + AxMinus\_nbr) \cdot T^{-1}$ 

```

2. Build T, Tinv from nor using the SAME GodunovFlux::BuildFrame (hpp:227) / BuildRotation (hpp:223) / BuildRotationInverse (hpp:219) helpers that BuildPerFaceFluxMatricesGlobal uses (keep the frame convention identical, or the central and Godunov face contributions will be inconsistent).
3. Face-local full Jacobian per side = flux_side.GetAxPlus() (hpp:178) + flux_side.GetAxMinus() (hpp:180). This MATCHES GodunovFlux::Central's exact construction (cpp:416), required for $\leq 1e-11$ byte-equivalence with Central. **(BUG-9)** Form centralSide = $0.5 \cdot T \cdot (AxPlus + AxMinus) \cdot Tinv$ using **two distinct scratch DenseMatrix objects** (tmp1, tmp2) and SetSize ONLY the output, exactly as cpp:282-295 does (mfem::Mult(A,B,C) requires C to be a separate, correctly-sized matrix from A/B). Apply the 0.5 either by folding it into (AxPlus+AxMinus) before the rotations or via centralSide *= 0.5 after; state which in the code comment. Do NOT alias output with a scratch.
4. The runtime apply reuses the existing BimaterialFlux::ApplyPerFaceFlux(centralSelf, centralNbr, Q_self, Q_nbr, F_h) (cpp:301-329, computes $F = M0 \cdot Q_self + M1 \cdot Q_nbr$) — no new apply function.

Interfaces

- New: BimaterialFlux::BuildPerFaceCentralMatricesGlobal(...) (signature above).
- Unchanged: ApplyPerFaceFlux.

Edge Cases to Handle

- **Acoustic guard (BUG-9).** The central builder uses GetAxPlus()+GetAxMinus() directly and does NOT route through BuildGodunovStateFaceLocal (the upwind projector), so it gets no automatic $\mu \leq \epsilon$ guard. Add an explicit MFEM_VERIFY(flux_self.mu() > eps && flux_nbr.mu() > eps, ...) (or read mu via the existing accessor) at the top of the builder. The GodunovFlux ctor already asserts $\mu > 0$; this adds the $> \epsilon$ SAFS-no-acoustic-region contract. Document as a follow-up that acoustic regions are unsupported.
- **Non-unit / sign of nor:** BuildFrame already normalizes; do not pre-normalize differently than the Godunov path.

Acceptance Criteria

- **Test 1.1** (CentralFlux_HomogeneousLimit_MatchesGodunovCentral): for ≥ 10 random materials $\times \geq 50$ random (Q_self, Q_nbr, nor), ApplyPerFaceFlux(centralSelf, centralNbr, ...) with flux_self==flux_nbr equals GodunovFlux(λ, μ, ρ).Central(nor, Q_self, Q_nbr, ...) to $\leq 1e-11$ relative.
- **Test 1.2** (CentralFlux_Consistency_PhysicalFlux): Q_self==Q_nbr==Q, one material = $F^* == A \cdot Q$ (compare to a GetAx-based (hpp:176) global face-normal Jacobian $\times Q$) to $\leq 1e-11$.
- **Test 1.3** (CentralFlux_HeterogeneousAveraging, BUG-3 primitive anchor): with $A_e1 \neq A_e2$, assert the builder uses the NEIGHBOUR Jacobian for the neighbour term — ApplyPerFaceFlux(cS, cN, Q, 0, F) == $\frac{1}{2} \cdot A_e1 \cdot Q$ and ApplyPerFaceFlux(cS, cN, 0, Q, F) == $\frac{1}{2} \cdot A_e2 \cdot Q$ to $\leq 1e-11$. (Guards against silently dropping A_nbr.)
- **Note (BUG-7):** the single-valuedness invariant ($F_h_e1 == F_h_e2$) is an *operator-level* property and is tested in Phase 3 (Test 3.2), NOT here. Do NOT add a primitive “swap (self,nbr) and states” test — it only checks commutativity of addition and would pass with the BUG-3 dispatch defect present.
- make test still green; no existing test perturbed.

Dependencies

- Depends on: nothing. Required by: Phase 2.

Phase 2: Precompute + store per-face central matrices on the matrix operator

Goal

BimaterialWaveOperator holds **side-symmetric** central matrices for exactly the faces in `central_flux_face_set_`, built from the SAME per-side flux operands as the Godunov matrices, for both interior and shared (cross-rank) faces.

Files to Modify

- `dynamic/bimaterial_wave_operator.hpp` — new member + helper declarations.
- `dynamic/bimaterial_wave_operator.inl` — factor operand resolution; add central precompute.

Detailed Requirements

1. Factor the ctor's per-face operand derivation into a private helper so the Godunov precompute (ctor) and the central precompute use byte-identical operands/orientation:

```
// side: 0 ⇒ Elem1 is "self"; 1 ⇒ Elem2 is "self". Used by the UPWIND build (per-side swap).
bool ResolveFaceFluxOperands_(int mesh_face, int side,
                              const GodunovFlux*& flux_self,
                              const GodunovFlux*& flux_nbr,
                              real_t nor_out[3]) const;
```

Refactor the existing ctor lambdas that build `per_face_bimaterial_flux_` (`bimaterial_wave_operator.inl:269-324`) to call this helper (pure refactor — Godunov matrices must remain byte-identical; guarded by the parity test 22/22).

2. **(BUG-3) Central storage is side-symmetric — ONE matrix pair per face, NOT per-side.** Store (`centralE1 =  $\frac{1}{2} \cdot A_{e1}$` , `centralE2 =  $\frac{1}{2} \cdot A_{e2}$`) keyed by mesh-face, with NO swap:

```
// [0]=centralE1 (mult. Q_e1), [1]=centralE2 (mult. Q_e2). Only central-set faces present.
std::unordered_map<int, std::array<mfm::DenseMatrix,2>> per_face_central_flux_;
```

(Memory-lean: central faces are ~5-10% of faces in Adjacent mode.) The same pair serves both sides; the dispatch (Phase 3) computes F^* once from (`centralE1`, `centralE2`, `Q_e1`, `Q_e2`) and copies it to side 2. Do **not** build a swapped side-1 pair.

3. Add private void `BuildPerFaceCentralFluxMatrices_()`; that clears `per_face_central_flux_`, then builds the pair for each central-set face. **Two arms (BUG-5):**

- **Interior faces:** iterate `central_flux_face_set_` (mesh-face indices); for each, derive (`flux_e1`, `flux_e2`, `nor`) with the `e1=self/e2=nbr` convention (NO swap), call `BimaterialFlux::BuildPerFaceCentralMatrices_` (`flux_e1`, `flux_e2`, `c[0]`, `c[1]`), store.
- **Shared (cross-rank) faces:** mirror the existing upwind Pass 2 (`inl:375-422`): iterate shared-face index `sf` (NOT `mesh_face`), skip fault shared faces (as `inl:393-397` does), map `sf` → `mesh_face` via `pmesh.GetSharedFace(sf)`, intersect with `central_flux_face_set_.count(mesh_face)`, look up the neighbour material in `shared_face_neighbour_material_[sf]` (`hpp:194`, keyed by `sf` — this is why the shared arm must loop `sf`, not `mesh_face`), build the side-0 pair ($\frac{1}{2} \cdot A_{e1_local}$, $\frac{1}{2} \cdot A_{e2_neighbour}$), store keyed by `mesh_face`. Keep the existing fail-loud `MFEM_VERIFY` for a missing bookkeeping entry (`inl:405-409`).
- The shared arm therefore needs either (a) the `sf`-loop above, or (b) a `mesh_face` → `sf` reverse index. Use (a) — it reuses the proven Pass-2 structure. (If `ResolveFaceFluxOperands_` is used for shared faces, extend it to accept an `sf` for the neighbour-material lookup; otherwise inline the shared lookup as Pass 2 does.)

4. **(BUG-6 / BUG-10 / BUG-15 / BUG-16) Lateral-heterogeneity guard — a declarative material.seam_continuous flag, NOT a runtime comparison or probe.** Neither runtime detector considered earlier is viable: (i) comparing the *stored* neighbour to local is a dead predicate (the stub sets `shared_face_neighbour_material_[sf] = local`, `inl:162` — always equal, BUG-10);

(ii) a material-field probe is unimplementable AND wrong — `MaterialField`'s only evaluator is `EvalAt(int elem, ElementTransformation&, IntegrationPoint&)` (`heterogeneous_material.hpp:183-209`), which has no transformation for a point in a neighbour element across a rank seam, and the coordinate-only `eval_at_xyz` lives on the `DepthProfile1DMaterial` wrapper (`heterogeneous_material.cpp:318-334`) that the operator does not hold (it stores only `const MaterialField* material_`, `hpp:182`) —

BUG-15; and an along-face-normal probe would false-abort TPV31, because for a fault-adjacent tet seam with $n_z \neq 0$ the two probe points sit at different depths and the legitimate `depth_profile_1d` variation (a full $\sim 28\%$ μ jump at a 2400/5000/10000 m discontinuity) trips any small tol — BUG-16. **The guard is therefore declarative:** abort any `Mode::Coefficient` *central shared* face unless the config declares `material.seam_continuous = true`. The message must name the seam and state that the cross-rank neighbour-material exchange is a local-side stub, so the user must affirm the material is seam-continuous (Constant or depth-only) or wait for the deferred `MPI_Allgather` exchange. Notes:

- This is deterministic and communication-free; it needs NO arbitrary-point material eval.
- It does NOT change the upwind `per_face_bimaterial_flux_build` (which keeps the stub verbatim $\Rightarrow$ the existing TPV31 ADER job stays byte-exact); it guards ONLY the new central build.
- **(BUG-24) The guard fires ONLY here, inside `BuildPerFaceCentralFluxMatrices_`, for faces in `central_flux_face_set_` with `mf_on_ true`.** With `mixed_flux=none` that set is empty (`wave_operator.inl:1653-1654`) so the guard is unreachable — the existing TPV31 ADER / `mixed_flux=none` parallel job is unaffected and stays byte-exact. The flag is read from `[material].seam_continuous` and set on the operator before `SetMixedFluxMode` (Phase 5 req 3).
- TPV31 sets `material.seam_continuous = true` — the same local-side-stub approximation the existing parallel TPV31 ADER bimaterial job already accepts.
- The fully-correct alternative (deferred) is the real `MPI_Allgather` of the peer `per_elem_lmr_`, which would fix BOTH paths and let the guard compare true-neighbour vs local. Cite R-004 (the stub limitation) when changing the WARNING at `inl:165-187` to this central-path abort.

5. Account the bytes in the existing precompute log + 4 GiB soft-cap warning.

Interfaces

- New private: `ResolveFaceFluxOperands_...`, `BuildPerFaceCentralFluxMatrices_()`.
- New member: `per_face_central_flux_` (side-symmetric, mesh-face-keyed).

Edge Cases to Handle

- **Empty central set** (`mixed_flux=none`): map stays empty; zero overhead; byte-exact.
- **Shared (cross-rank) fault-adjacent face:** only side 0 is populated (`Elem1` local), consistent with how `SharedInteriorFaceFlux_` is dispatched (`mesh_face` passed in; `bimaterial_wave_operator.inl:471-497`). Neighbour material from `shared_face_neighbour_material_` keyed by `sf` (see req 3, BUG-5).
- **Laterally-varying neighbour material:** abort (req 4, BUG-6/BUG-10/BUG-15/BUG-16) via the declarative **material.seam_continuous flag** — NOT a stored-neighbour-vs-local compare (always equal by construction, `inl:162`) and NOT a material-field probe (no arbitrary-point evaluator; conflates depth with lateral variation). A `Mode::Coefficient` *central shared* face without `seam_continuous = true` aborts; with it, builds.

Acceptance Criteria

- ☐ Ctor refactor leaves `per_face_bimaterial_flux_` byte-identical (`test_bimaterial_wave_operator_parity` 22/22 unchanged).
- ☐ After `SetMixedFluxMode(Adjacent)` on a het fixture with a fault, `per_face_central_flux_size ==` number of central-set mesh-faces (one pair each), disjoint from fault faces.
- ☐ Each stored pair satisfies `c[0] == ½·A_e1` and `c[1] == ½·A_e2` (no swap) — checked indirectly by the Phase 3 single-valuedness test.

Dependencies

- Depends on: Phase 1. Required by: Phase 3.

Phase 3: Dispatch + lift the mutual-exclusion abort

Goal

`BimaterialWaveOperator` accepts `Adjacent/AllContinuous`, building the central set + matrices, and dispatches central vs bi-material-Godunov per face — depositing a single-valued F^* on central faces.

Files to Modify

- `dynamic/bimaterial_wave_operator.inl` — rewrite `SetMixedFluxMode`, `InteriorFaceFlux_`, `SharedInteriorFaceFlux_`.
- `dynamic/bimaterial_wave_operator.hpp` — update the stale “aborts on enable” doc comment.

Detailed Requirements

1. **(BUG-1) Replace the ACTUAL abort — the override body at `bimaterial_wave_operator.inl:567-576`** (the `MFEM_VERIFY(m == MixedFluxMode::None, ...)`), NOT the comment at `wave_operator.inl:1623-1626`:

```
void BimaterialWaveOperator<MeshType>::SetMixedFluxMode(MixedFluxMode m) {
    WaveOperator<MeshType>::SetMixedFluxMode(m);    // base: validate, Allgather/Allreduce mode
                                                    // check (inl:1593-1610), BuildCentralFlux-
                                                    // FaceSet_, set mf_on_/mixed_flux_mode_
    BuildPerFaceCentralFluxMatrices_();              // Phase 2 helper (clears first ⇒ idempotent)
}
```

- Keep the base’s `Adjacent ⇒ fault_attr>0` verify and the rank-consistency `Allreduce`.
- Keep the base’s `use_precomputed_face_fluxes_` mutual-exclusion (false here, but defensive).
- **(BUG-1) Update ALL stale comments** the old structure left behind, none of which the Rev-1 plan named: `bimaterial_wave_operator.inl:518-520` (inside `ComputeMaxDt`: “Mixed flux is disabled on the matrix path ... `mixed_flux_mode_` is always `None` here”), the `hpp` doc comment at `bimaterial_wave_operator.hpp:121` (“Mixed flux is scalar-only (R-003): abort on any non-`None` mode”), and the base comment at `wave_operator.inl:1623-1626`. Cite **R-003** where the structural exclusion is lifted (CLAUDE.md “never revert a previous fix without justification”; justification = `drdg3d` oracle, this feature’s purpose).

2. **(BUG-3) `InteriorFaceFlux_` — add the single-valued central branch BEFORE the Godunov apply:**

```
if (mf_on_ && central_flux_face_set_.count(mesh_face) > 0) {
    const auto& c = per_face_central_flux_.at(mesh_face);    // .at() ⇒ loud miss (BUG-3 store)
    BimaterialFlux::ApplyPerFaceFlux(c[0], c[1], Q_self, Q_nbr, F_h_e1); //
    ↪ ½A_e1·Q_e1+½A_e2·Q_e2
    for (int comp = 0; comp < 9; ++comp) F_h_e2[comp] = F_h_e1[comp];    // single-valued copy
    phaser_dispatch_count_ += 2; return;
}
// ... existing bi-material Godunov apply (per-side, unchanged) ...
```

The runtime passes the UNSWAPPED (`Q_self=Q_e1`, `Q_nbr=Q_e2`) to both sides (confirmed `inl:471-484`), and the side-symmetric store (Phase 2) means side 2 is the exact copy of side 1.

3. **`SharedInteriorFaceFlux_` — same central branch, single `F_h` (side 0 only):**

```
if (mf_on_ && central_flux_face_set_.count(mesh_face) > 0) {
    const auto& c = per_face_central_flux_.at(mesh_face);
    BimaterialFlux::ApplyPerFaceFlux(c[0], c[1], Q_self, Q_nbr, F_h);
    ++phaser_dispatch_count_; return;
}
```

4. Use `.at()` (not `operator[]`) so a missing central matrix on a central-set face aborts loudly rather than inserting a zero matrix.

Edge Cases to Handle

- **`mixed_flux=none`:** `mf_on_ false` ⇒ central branch never taken ⇒ byte-exact Godunov path.
- **Re-entrant `SetMixedFluxMode`** (mode changed twice): `BuildPerFaceCentralFluxMatrices_` clears first; idempotent.

Acceptance Criteria

- ☐ `interior_flux="matrix" + mixed_flux="adjacent"` no longer aborts at `SetMixedFluxMode`.
- ☐ `mixed_flux=none` matrix path byte-identical (parity test 22/22 unchanged).
- ☐ **Test 3.1** (`Dispatch_CentralVsGodunov_PerFaceSelection`): on a het fixture, central-set faces produce the central F^* (matches the Phase-1 primitive) and non-central interior faces produce the bi-material Godunov F^* ; verify via direct value comparison and `phaser_dispatch_count_ deltas`.

- **Test 3.2** (Dispatch_CentralFlux_SingleValued_UnderHeterogeneity, BUG-3/BUG-7 — the load-bearing guard): on a two-element fault-adjacent fixture with a GENUINE contrast ($\mu_{e1} \neq \mu_{e2}$), InteriorFaceFlux_ on a central-set face gives $F_{h_{e1}} = F_{h_{e2}}$ componentwise to $\leq 1e-12 \cdot \text{scale}$. **Must use heterogeneous material** — it passes trivially under homogeneity. (Fails on the Rev-1 swapped-storage design; passes with the Phase-2 side-symmetric store.)
- **Test 3.3** (Dispatch_None_ByteExactGodunov): `mixed_flux=none` $\Rightarrow$ central branch never taken $\Rightarrow$ byte-identical to the pre-merge bi-material Godunov path.
- **Test 3.4a** (MPI, `np=2`, `tests/parallel/`, `Dispatch_SharedCentralFace_Build`, BUG-5): a shared fault-adjacent face in Adjacent mode with `material.seam_continuous=true` builds + dispatches the side-0 central matrices; value matches the Phase-1 primitive.
- **Test 3.4b** (MPI, `np=2`, `Dispatch_SharedCentralFace_RequiresSeamContinuous`, BUG-6/BUG-10/BUG-15): a `Mode::Coefficient` central shared face **WITHOUT** `material.seam_continuous=true` **aborts** with the seam-named message; **WITH** it, builds. Written against the declarative flag (the only viable mechanism) — NOT a probe or a neighbour vs local compare.
- **Test 3.4c** (MPI, `np=2`, `Dispatch_DepthProfile_NoFalseAbort`, BUG-16): a depth-varying (`depth_profile_1d`) material with `seam_continuous=true`, on a fixture whose partition-seam normal has a z-component, **builds (does NOT abort)** — proving the gate does not false-trip on legitimate depth variation. (An along-normal material probe would wrongly abort here.)

Dependencies

- Depends on: Phase 2. Required by: Phases 4, 5.

Phase 4: CFL de-rating on the matrix path

Goal

`BimaterialWaveOperator::ComputeMaxDt` applies the same mixed-flux CFL factor as the scalar path, from a single shared source of truth — with the `cfl_rk_aware_gating` it currently lacks.

Files to Modify

- `dynamic/wave_operator.hpp` / `.inl` — extract the factor switch into a protected helper.
- `dynamic/bimaterial_wave_operator.inl` — DELETE the divergent local switch; apply the shared factor in the per-element CFL walk.

Detailed Requirements

1. Extract the switch (`mixed_flux_mode_`) block at `wave_operator.inl:5880-5903` into a protected method on the base:

```
protected: real_t MixedFluxCflFactor() const;
//   None           → 1.0
//   Adjacent       → cfl_rk_aware_ ? 0.6 : 0.9
//   AllContinuous  → cfl_rk_aware_ ? 0.7 : 0.4
//   default        → MFEM_ABORT (R-1600)
```

Scalar `ComputeMaxDt` calls it (pure refactor, byte-exact for the scalar operator — verified the factors/gating match exactly, check Item 2).

2. **(BUG-2) DELETE the bi-material operator's OWN switch at `bimaterial_wave_operator.inl:521-531`** (which carries `Adjacent→0.9`, `AllContinuous→0.4` with **no `cfl_rk_aware_gating`** — divergent from scalar) and its hardcoded `cfl_mixed_flux_factor`. Multiply the per-element walk's `dt_e` (`inl:546`) by `MixedFluxCflFactor()` instead. Keep the per-element heterogeneous walk (`inl:539-548`) and the `MPI_Allreduce(MIN)` (`inl:553-555`).

- **This is NOT byte-exact for the bi-material operator** and the plan states so explicitly: today those `Adjacent/AllContinuous` arms are dead code (unreachable behind the abort) and use the ADER factors unconditionally; after the merge, with `cfl_rk_aware_true` (the required RK path), the factors become `0.6/0.7` — the *intended* behaviour. There is no existing test to violate (mixed flux was disabled here), so the byte-exact regression contract is not touched; the change is to previously-unreachable code.

3. **(BUG-4 cross-ref) Enforce the integrator requirement:** if `mixed_flux_mode_ != None` and `!cfl_rk_aware_`, abort with the ADER-instability message. Put this in the shared helper (or in both `ComputeMaxDts`) so neither operator can run central+ADER. Keep it consistent with the driver-level guard relaxed in Phase 5 (same condition, same message).

Acceptance Criteria

- ☐ Scalar `ComputeMaxDt` byte-identical after refactor (existing CFL tests pass).
- ☐ **Test 4.1** (`ComputeMaxDt_FactorMatchesScalar_AllModes`, BUG-2): for `mode ∈ {None, Adjacent, AllContinuous}` × `cfl_rk_aware_ ∈ {false, true}`, `BimaterialWaveOperator` and `WaveOperator` return the SAME `MixedFluxCflFactor_()` (`None`→1, `Adjacent`→0.6/0.9, `AllContinuous`→0.7/0.4). Confirms one source of truth; no residual local switch.
- ☐ **Test 4.2** (`ComputeMaxDt_CentralPlusADER_Aborts`): `mixed_flux != None` and `!cfl_rk_aware_` aborts with the instability message on BOTH operators.

Dependencies

- Depends on: Phase 3. Required by: Phase 5.

Phase 5: Driver + config wiring (lift ALL series guards; wire `seam_continuous`)

Goal

`spatial_dyn_driver` runs `interior_flux="matrix" + mixed_flux="adjacent"/"all_continuous" + RK` end-to-end. This requires relaxing **every** mutual-exclusion guard in the series — an exhaustive sweep (`interior_flux/mixed_flux/InteriorFlux/MixedFluxMode/is_rk` across drivers/ + `spatial/code/` + `dynamic/`) found exactly THREE blocking guards in three files — and wiring the new `[material].seam_continuous` flag through the config parser.

Guard inventory (exhaustive — relax ALL THREE; do NOT touch the rest)

THREE guards block `matrix + mixed_flux!="none"`, in firing order: - **G1 (BUG-21) — `spatial/code/spatial_friction.cpp:1167`** (config parse; fires FIRST inside `LoadSpatialFrictionConfig`, called at `spatial_dyn_driver.cpp:616` — UPSTREAM of the CLI override and every later guard): `MFEM_VERIFY(interior_flux==Scalar || mixed_flux=="none", "matrix incompatible with mixed_flux ...")`. **RELAX** (req 1). - **G2 (BUG-4) — `spatial_dyn_driver.cpp:809-816`** (driver, AFTER the CLI integrator override at :679): `MFEM_VERIFY(!is_rk || interior_flux==Scalar, ...)`. **RELAX** (req 2). - **G3 (BUG-1) — `bimaterial_wave_operator.inl:569-574`** (operator): `MFEM_VERIFY(m==None, ...)`. **REPLACE** (done in Phase 3 step 1). These guards are CORRECT and KEPT — the sweep confirmed TPV31 (`matrix + depth_profile_1d + RK`) does NOT trip them; do NOT modify them: - `spatial_friction.cpp:1702-1707` — `matrix` requires a non-Constant material (TPV31 is `depth_profile_1d` (ok)). - `spatial_dyn_driver.cpp:1078-1085` — `scalar` requires a Constant material (TPV31 is `matrix` (ok)). - `wave_operator.inl:604-610` (`UsePrecomputedFaceFluxes`) + :1613-1616 (base `SetMixedFluxMode`) — the R-1203 precomputed-flux ↔ mixed-flux mutual exclusion; `use_precomputed_face_fluxes_` is false on this path (ok) (keep defensive). - The central+ADER abort (Phase 4 `ComputeMaxDt` + the new G2 ADER arm) — intended.

Files to Modify

- `spatial/code/spatial_friction.{hpp,cpp}` — **(BUG-22)** relax G1; add a `bool seam_continuous` field to `MaterialSpec` (`spatial_friction.hpp:573`) and parse it in the `[material]` block (`spatial_friction.cpp:1626+`). **This is a deliberate source-edit scope expansion** (see Constraints): the TPV*/TPV31 spatial configs all parse through this file (BP5 does not — native driver), so it MUST carry a parser-regression test (Test 5.3c).
- `drivers/spatial_dyn_driver.cpp` — relax G2; read `[material].seam_continuous` and pass it to the operator (ctor arg / `SetSeamContinuous(bool)`) BEFORE `SetMixedFluxMode`; log the combination; update the stale comments.

Detailed Requirements

1. **(BUG-21) Relax G1 at spatial/code/spatial_friction.cpp:1162-1167.** It rejects matrix + mixed_flux!="none" at config-load — upstream of everything — so the Phase-7 TOML (mixed_flux="adjacent") aborts before any operator is built. Remove the mutual exclusion (matrix + mixed is now supported) and update the "scalar-path-only / R-1203 sibling" comment at :1160-1167. Do NOT gate on the integrator here — the final integrator is unknown until the CLI override (driver :679); the integrator constraint is enforced downstream at G2 (req 2) + Phase 4 ComputeMaxDt.
2. **(BUG-4) Relax G2 at spatial_dyn_driver.cpp:809-816** (currently MFEM_VERIFY(!is_rk || interior_flux==Scalar, ...)). Rewrite to: **ALLOW** matrix + RK when mixed_flux != none; **REJECT** matrix + ADER + mixed_flux != none with the ADER-instability message (same condition + message as the Phase-4 ComputeMaxDt guard); matrix + RK + mixed_flux == none **allowed by default** (RK on the bimaterial upwind operator is harmless — only central flux needs RK). Update the now-stale comment at :806-808.
3. **(BUG-22) Add + wire [material].seam_continuous.** Add bool seam_continuous = false; to MaterialSpec (spatial_friction.hpp:573); parse toml_bool(m, "seam_continuous", false) in the [material] block (spatial_friction.cpp:1626+, alongside kind/depth_axis). In the driver, read cfg.material.seam_continuous and call wave.SetSeamContinuous(...) (or pass via the matrix ctor) **BEFORE** SetMixedFluxMode (:1245) so the Phase-2 guard reads the correct value, not the default. Default false ⇒ all existing configs (which omit the key) are unchanged.
4. At the SetMixedFluxMode call (line 1245), add a rank-0 log line when both are active: "[mixed-flux] matrix (bi-material) + <mode> central flux enabled".
5. Confirm the operator-selection block (1099-1138): scalar branch at 1100, matrix branch at 1110, matrix requires material.mode != Constant (1123-1128). No change needed there; note it.
6. Confirm SetCflRkAware(true) (line 1902) runs before ComputeMaxDt (line 1905) on the RK path, so the Phase-4 factor sees cfl_rk_aware_ == true. Confirm the RK Butcher tableau selection at 2660 is reached. No --dt override is added (auto-CFL; see Phase 7).
7. Do NOT change defaults: interior_flux default stays scalar, mixed_flux default none, time_integrator default ADER, seam_continuous default false.

Acceptance Criteria

- ☐ **Test 5.1** (Driver_MatrixPlusAdjacentPlusRK_ReachesTimeLoop, BUG-4/BUG-21): a --dry-run / scripted parse with interior_flux=matrix + mixed_flux=adjacent + --time-integrator rk4 (+ non-Constant material + seam_continuous=true) parses past G1 (spatial_friction.cpp:1162) AND reaches the time loop past G2 (driver:811) WITHOUT abort. (Fails today at BOTH; passes after relaxing both.) Parse/dry-run on a tiny fixture, NOT the TPV31 mesh.
- ☐ **Test 5.2** (Driver_MatrixPlusAdjacentPlusADER_Aborts): same with --time-integrator ader aborts with the instability message (at G2 / Phase-4).
- ☐ **Test 5.3** (NEW, BUG-21/BUG-22/BUG-25): (a) LoadSpatialFrictionConfig on a matrix + adjacent TOML no longer aborts at spatial_friction.cpp:1162; (b) [material].seam_continuous round-trips into MaterialSpec; (c) **parser-regression** — extend the existing tests/unit/test_tpv_config_parse.cpp (which already loads tpv205/102/104_spatial.toml and asserts FIELD-BY-FIELD via TEST_ASSERT) to ALSO load tpv31/configs/tpv31.toml (+ p2/p3) and assert, field-by-field, that seam_continuous defaults false and every previously-asserted field is unchanged. There is no SpatialFrictionConfig::operator==/dump, so "byte-identical" means this field-by-field check — NOT a struct-equality call. **Do NOT put BP5 in the parse corpus** — BP5 has no spatial TOML and never invokes this parser (its byte-exactness is protected by the edit being purely additive to a parser BP5 never calls).
- ☐ TPV102/104/205 + BP5 + the existing TPV31 ADER driver invocations unchanged (defaults untouched) — byte-exact.

Dependencies

- Depends on: Phase 4. Required by: Phases 6, 7.

Phase 6: Unit tests, homogeneous-equivalence gate, Makefile

Goal

The combination is covered by unit tests, the homogeneous-limit equivalence gate is green, and the new targets are fully registered in `make test`.

Files to Create

- `tests/unit/test_bimaterial_central_flux.cpp` (Phase 1 — Tests 1.1-1.3).
- `tests/unit/test_bimaterial_mixed_flux_dispatch.cpp` (Phase 3 — Tests 3.1-3.3).
- `tests/parallel/test_bimaterial_mixed_flux_shared.cpp` (Phase 3 — Test 3.4, `np=2`).
- `tests/unit/test_bimaterial_mixed_flux_cfl.cpp` (Phase 4 — Tests 4.1-4.2).
- `tests/unit/test_bimaterial_mixed_flux_homog_equivalence.cpp` (Phase 6 — Test 6.1, **the gate**).

Files to Modify

- `Makefile` — register the new unit targets (see BUG-8 recipe below).

Detailed Requirements

1. **Homogeneous-equivalence gate (Test 6.1):** build the SAME small mesh + homogeneous material twice — once as `WaveOperator + SetMixedFluxMode(Adjacent)`, once as `BimaterialWaveOperator` constructed DIRECTLY from a homogeneous `MaterialField::MakeCoefficient(&lam_c, &mu_c, &rho_c)` (NOT via the driver — the driver's `material.mode != Constant` guard at `driver:1123` would reject a Constant material; construct the operator directly as the parity test does, `test_bimaterial_wave_operator_parity.cpp:254-255`) + `SetMixedFluxMode(Adjacent)`. Apply Mult to ≥ 3 random states; assert residuals $\leq 1e-9$. Repeat for `AllContinuous`.
 - **NOTE:** this gate passes EVEN WITH the BUG-3 defect ($\text{homogeneous} \Rightarrow A_{e1} = A_{e2}$). It is necessary but NOT sufficient; **Test 3.2 (het single-valuedness) is the sufficient guard**.
2. **(BUG-8) Makefile registration — FOUR coordinated edits per test, plus the correct object-dependency list:**
 - `TEST_<NAME>_SRC` variable (pattern `makefile:311-317`).
 - `TEST_<NAME>_OBJ` derivation.
 - A link target with the right deps:
 - **Primitive** (`test_bimaterial_central_flux`): links `GODUNOV_FLUX_BIMATERIAL_OBJ` + `GODUNOV_FLUX_OBJ` only (cf. `seas_test_godunov_central_flux, makefile:5248`).
 - **Operator-level** (`*_dispatch`, `*_cfl`, `*_homog_equivalence`): link the full `WAVE_OPERATOR_OBJ` + `GODUNOV_FLUX_BIMATERIAL_OBJ` + `PRECOMPUTED_FACE_FLUXES_OBJ` + `GODUNOV_FLUX_OBJ` + `PML_LAYER_OBJ` + `FAULT_FACE_FLUX_OBJ` + `FRICTION_SOLVER_OBJ` (cf. `makefile:1638, 5252`).
 - A test-`<name>`: run alias.
 - Inclusion in the aggregate `make test` group list (`makefile:859-893`).
3. **Heterogeneous local smoke fixture (feeds Phase 7's local gate, NOT a production run):** a tiny ($\leq$ a few hundred elements) two-material fixture + TOML under `tests/` for an $np \leq 4$ local smoke that exercises a few steps of matrix + adjacent + rk4. This is the largest thing run locally; the real TPV31 mesh is Frontera-only (Phase 7).

Acceptance Criteria

- ☐ All new unit tests pass; `make test` green; primitive vs operator object-dep lists correct.
- ☐ Homogeneous-equivalence gate passes for `Adjacent` AND `AllContinuous` at $\leq 1e-9$.
- ☐ Test 3.2 (het single-valuedness) passes — the BUG-3 structural guard.
- ☐ $np \leq 4$ local het smoke advances ≥ 5 steps without NaN / `dt→0`.
- ☐ Parity + byte-exact suites unchanged (TPV*/BP5/bimaterial-none/TPV31-ADER).

Dependencies

- Depends on: Phases 1-5. Required by: Phase 7.

Phase 7 (FINAL): TPV31 heterogeneous end-to-end validation on Frontera

Goal

A complete, ready-to-submit Frontera job for **SCEC TPV31** running the merged `interior_flux="matrix"` (depth-dependent bi-material) + `mixed_flux="adjacent"` central flux + RK45 path, with a real-mesh dispatch gate, a SeisSol reference overlay, and the project's Frontera safeguards — staged for the user to run (NOT auto-submitted).

Why TPV31

TPV31 (`tpv31/configs/tpv31.toml`) is `kind="depth_profile_1d"` (a `Mode::Coefficient`, vertically-layered bi-material), so it MUST use `interior_flux="matrix"` (the scalar Godunov ctor is mutually exclusive with `Coefficient` material — see the config comment `tpv31.toml:236-240`). The existing TPV31 Frontera jobs (`jobs/tpv31_spatial/tpv31_p2_ader03_noflux_50m_normal.sbatch`) run **ADER + mixed_flux=none**. The existing mixed-flux jobs (TPV104/102, `jobs/tpv104_spatial/tpv104_p2_rk45_mixedflux_12N_600r_normal.sbatch`) run **RK45 + scalar** (homogeneous). **TPV31 + RK45 + mixed_flux=adjacent is the first job to combine matrix-bimaterial + central flux** — the exact path Phases 1-6 build. It is therefore the natural end-to-end target.

(BUG-26) Friction law / RK stepper: TPV31 is `law="slip_weakening"` (LSW; `tpv31.toml:58,77`, [`friction.slip_weakening`] :270, `instantaneous overstress_circular nucleation` :304) — NOT rate-and-state. So on the RK path the driver dispatches to `AdvanceRKCoupledLSW_Spatial` (`spatial_dyn_driver.cpp:2864,2873`; precondition `GetFaultFrictionLaw()==LSW`, `rk_time_stepper.hpp:389`), not the rate-state `AdvanceRKCoupled_Spatial`. This does NOT affect the feature: the central-flux dispatch lives in the law-agnostic wave.Mult (Phase 3), exercised identically by both coupled-RK steppers. The Phase-6 local smoke + Test 5.1 fixture should be LSW (or any law) — the central path is law-independent. The existing TPV31 ADER job already runs LSW + matrix.

Files to Create

- `tpv31/configs/tpv31_rk_mixedflux.toml` — copy `tpv31/configs/tpv31.toml` and change TWO keys: [`numerics`].`mixed_flux` "none" → "adjacent", and add [`material`].`seam_continuous` = `true` (required by the Phase-2 req-4 guard — TPV31 is depth-only, so the local-side-stub neighbour material is acceptable, the same approximation the existing parallel TPV31 ADER bimaterial job already runs under). The key is added to `MaterialSpec` and parsed in `spatial/code/spatial_friction.{hpp,cpp}` (Phase 5 req 3 — an in-scope edit, BUG-22), then the driver reads `cfg.material.seam_continuous` and sets it on the operator (`SetSeamContinuous` / ctor) BEFORE `SetMixedFluxMode`; the Phase-2 guard reads that member. Everything else identical: `interior_flux="matrix"`, [`mesh`].`order`=1, the `depth_profile_1d` material, stress, friction, nucleation, `tfinal="15s"`, `cfl`=0.5, `cfl_safety="dg"`. (Optional p2 sibling `tpv31_p2_rk_mixedflux.toml` from `tpv31_p2.toml` for a production-scale run.) The time integrator is selected on the CLI (`--time-integrator rk45`), mirroring the TPV104 mixed-flux job, so the resolved scheme prints in the startup banner and cannot silently drift.
- `jobs/tpv31_spatial/tpv31_p1_rk45_mixedflux_50m_normal.sbatch` — the Frontera job.
- `tpv31/test_visualize_results_tol.py` — **Tests NEW-7.1 + NEW-7.4 + NEW-7.5**: the new `--tol-rms/--tol-peak` gate. NEW-7.1 — MFEM .dat and reference .txt on OFFSET time grids (exercises `np.interp`); `>band` = `exit≠0`, `within-band` = `exit 0`. NEW-7.4 — a zero-crossing reference (pre-nucleation zeros + a slip-rate pulse) does NOT spuriously fail under the peak-normalized metric. NEW-7.5 (BUG-23) — a truncated MFEM trace (covers far less than the reference span) ⇒ non-zero exit with an explicit insufficient-coverage failure, NOT a silent pass. Runs via `pytest` (no mesh).

Files to Modify

- `tpv31/visualize_results.py` — **(BUG-11)** add a quantitative `--tol-rms/--tol-peak` pass/fail gate (it is plot-only today; the flags do not exist). It is a benchmark tool, in-scope to edit (NOT under the source no-touch tree). See req 5(a).
- `tpv31/benchmark_data/README.md` — **(BUG-12)** correct the stale reference-data description (it claims the dir is empty and .dat-format while 60 .txt files are committed). See req 5(b).

Detailed Requirements

- sbatch — mirror the two proven templates** (`tpv104_p2_rk45_mixedflux_12N_600r_normal.sbatch` for the RK/mixed-flux CLI, `tpv31_p2_ader03_noflux_50m_normal.sbatch` for the TPV31 mesh/preflight struc-

ture). **Copy the module + LD_LIBRARY_PATH block VERBATIM** (project memory: match-working-sbatch-modules — do not omit fftw3 etc.):

```
module load intel/19.1.1
module load impi/19.0.9
module load hypre/2.31.0
module load mumps/5.3
module load parmetis
module load petsc/3.15
module load fftw3/3.3.8
export LD_LIBRARY_PATH="${TACC_HYPRE_LIB}:${LD_LIBRARY_PATH:-}"
```

Keep the repo-root walk-up, the seas_spatial_dyn_driver existence check, and the Gmsh-v2.2 mesh regen (gmsh -format msh22 -3 tpv31/mesh/tpv31_50m.geo -o tpv31/mesh/tpv31_50m.msh).

2. **Pre-flight greps** (fail-loud, mirror the TPV31 ADER job): assert the new TOML sets interior_flux = "matrix" (config-only, no CLI override) and mixed_flux = "adjacent". Assert [mesh].order matches the job name (p1 ⇒ order = 1).
3. **CLI** (re-assert the scheme so it appears in the startup banner + --verify-dispatch; see req 4):

```
ibrun ./seas_spatial_dyn_driver \
--config tpv31/configs/tpv31_rk_mixedflux.toml \
--mesh tpv31/mesh/tpv31_50m.msh \
--time-integrator rk45 \
--mixed-flux adjacent \
--tfinal 15 \
--paraview --paraview-fault-vtu --paraview-fault-dt 0.1 --paraview-max-snapshots 300 \
--print-derived \
--checkpoint-every 2000 \
--output-dir "${OUT}" \
${RESTART_FLAGS} \
--verify-dispatch
```

- **Do NOT pass --dt** (project memory: explicit-RK CFL — omit --dt, use the driver's auto-CFL; the RK-aware Adjacent factor 0.6 from Phase 4 + cfl=0.5 + cfl_safety="dg" set the stable dt). Read the resolved/empirical dt off --print-derived and the *.out to size the restart chain. The mixed-flux + matrix CFL factor is an empirical STARTING calibration (check Risk Assessment; wave_operator.inl §14.4) — flag, do not claim "validated."
 - **--deriv-cache: OMIT** for the FIRST TPV31 run as a deliberate first-run simplification — **NOT because it is unsafe (BUG-14)**. The driver comment at spatial_dyn_driver.cpp:1140-1148 states the cache "works for both the scalar and bimaterial (matrix) operators (the cache is geometry-only)", is round-off-only (R-002) and budget-guarded (R-004, ~18 MB/rank at p2, ~72 MB/rank at p3 for TPV31); :1149-1158 enables it unconditionally with no bimaterial rejection. The reasons to omit it here are (a) isolate the CFL calibration with one fewer variable, and (b) it accelerates the ADER hot path while this job is RK45, so the benefit is limited. Re-enable it once the CFL is calibrated. Document this (correctly) in the sbatch comment.
 - Include the restart-chain block (per-job-ID OUT, TPV31_RESTART resume) verbatim from the TPV31 ADER job (the 15 s run spans multiple 48 h segments).
4. **Real-mesh dispatch gate (the end-to-end gate for BUG-3 + BUG-4)**. Because of the no-local-run rule, the FIRST execution of the merged matrix + RK + adjacent dispatch on the real TPV31 mesh happens on Frontera (idev or the head of the normal job). The gate is: the run reaches the time loop and the resolved scheme appears in the *.out. **(BUG-13) The three scheme lines come from TWO sources, not all from --verify-dispatch:** the unconditional rank-0 startup banner prints time integrator: (spatial_dyn_driver.cpp:853) and mixed flux: (:859) with multi-space padding (e.g. mixed flux: adjacent), while the --verify-dispatch block (:2130-2147) prints [verify-dispatch] interior flux : matrix (heterogeneous bimaterial Riemann) (note the space before the colon and the descriptive value, not a bare matrix). Grep the *.out with **tolerant, whitespace-insensitive regex** for time integrator:, mixed flux:, and interior flux — mirroring the proven TPV104/TPV31 sbatch banner-grep style — NOT brittle exact strings like interior flux: matrix. This proves the Phase-5 guard relaxation and the Phase-2/3 dispatch resolve on a production mesh. A short idev slice (small --tfinal, e.g. a few hundred steps) is the recommended gate before the 48 h run.
 5. **SeisSol reference overlay (quantitative validation)**. After the run, compare the on-fault station traces in \${OUT} against the committed SCEC SeisSol reference (tpv31/benchmark_data/scec_seisSol/

30 .txt files) via `tpv31/visualize_results.py`. **(BUG-11) The `--tol-rms/--tol-peak` flags do NOT exist today** — the script is plot-only (its `argparse` is `--eqdyna/--seisol/--both/--no-benchmark/--stations/--save/--output-dir/--closeup-t`, with no error metric and no pass/fail exit). So this requirement INCLUDES implementing them:

- (a) **Extend `tpv31/visualize_results.py`** with `--tol-rms <x> --tol-peak <y>` computing a well-defined, zero-safe gate (reuse the reader `load_reference_file / _parse_numeric_table` at `visualize_results.py:158-192`, which parses the committed 8-column .txt and flips n-stress to compression-positive). Three things the metric MUST specify:
 - **Resample + overlap window (BUG-19/BUG-23):** the script does NOT interpolate today (no `np.interp`; the plot path at :235-260 draws each dataset on its own time grid). MFEM (adaptive RK45, no `--dt`) and the SeisSol .txt (fixed cadence) are on different samples, so define the overlap window explicitly — `t0 = max(mfem_t[0], ref_t[0])`, `t1 = min(mfem_t[-1], ref_t[-1])` — restrict BOTH series to `[t0, t1]`, and **interpolate the MFEM channel onto the reference samples within `[t0, t1]` via `np.interp` BEFORE** any difference. Do NOT interpolate onto the FULL reference grid: `np.interp` clamps out-of-range to the endpoint, so a truncated MFEM run would flat-line its tail and spuriously FAIL.
 - **Minimum-coverage guard (BUG-23):** the gate is only meaningful on a completed run. If `(t1 - t0)` is less than a stated fraction of the reference span (e.g. `< 90%`, or `t1 < t_rupture_arrival`), **fail with an explicit “insufficient coverage” message and a non-zero exit** — do NOT silently pass over a tiny early-time window (a wall-truncated / restart-failed MFEM run must not look like agreement). Covered by Test NEW-7.5.
 - **Named channels (BUG-18):** gate the SCEC on-fault quantities BY NAME — `V_strike`, `slip_strike`, `tau_strike` (and the `_dip` channels only if the station has dip motion), plus `sigma_n`. **Exclude `mu_eff`** (MFEM col 8): its reference is all-NaN (reader :191), so a NaN error compares false and would *silently pass*. Use the column map at `visualize_results.py:145-155`.
 - **Peak-normalized metric (BUG-17):** per gated channel, `peak_rel = max_t|mfem(t)-ref(t)| / max(max_t|ref(t)|, floor)` and `rms_rel = sqrt(mean_t (mfem-ref)2) / max(sqrt(mean_t ref2), floor)`, with a small denominator `floor` — NOT a per-sample `|mfem-ref|/|ref|`, which blows up at the pre-nucleation zero-crossings. Print a per-station × per-channel table and **exit non-zero** if ANY gated channel of ANY station exceeds `--tol-peak` or `--tol-rms`. Pass-band ~5-10% peak (SeisSol is a different code). Covered by Test NEW-7.1 (offset time grids = exercises `np.interp`) and Test NEW-7.4 (zero-crossing reference = no spurious failure).
- (b) **(BUG-12) Fix the stale `tpv31/benchmark_data/README.md`** to match reality: 60 .txt files (`scec_seisol/`, `scec_eqdyna/`), the SeisSol multi-line comment header, the 8 numeric columns (`h=strike`, `v=dip`, n-stress compression-negative), and the `tpv31_{code}_x2*_x3_*.txt` glob the script uses (`visualize_results.py:201,575`). Remove the contradictory “directory is otherwise empty” / .dat-format sentences. Cite the script’s actual parser contract — not the stale README — as the format authority. Document this as the quantitative bi-material rupture validation (transmission/reflection at the depth contrasts), gated by the new exit-code, not a manual eyeball.

6. **Approval + submission policy (project memory: ask-before-Frontera-runs).** The implementer creates the config + sbatch and verifies them locally (pre-flight greps, shellcheck, the parse-level Test 5.1), then **STOPS and notifies the user** with the exact sbatch command. Do NOT submit any Frontera job, run `idev`, `ssh`, or check the allocation without explicit user approval. “We will run the tests on Frontera” = the USER submits after reviewing the staged job.

Edge Cases to Handle

- **dt calibration:** `central+RK+matrix CFL` is a starting calibration; if the *.out shows `dt→0` or `V_max` runaway, recalibrate `cfl / the Phase-4 factor` on Frontera — not a local pass/fail.
- **Parallel seam material (BUG-6/BUG-10/BUG-15/BUG-16):** TPV31’s `depth_profile_1d` is depth-only, so the local-side-stub neighbour material is acceptable — the same approximation the existing parallel TPV31 ADER bimaterial job already runs under. The merged config sets `[material].seam_continuous = true`, which satisfies the Phase-2 req-4 **declarative** guard. There is NO runtime material probe (it would false-abort on the depth discontinuities at 2400/5000/10000 m — BUG-16). Tests 3.4b/3.4c (`np=2`) are the local guards. Note this in the sbatch comment.
- **Mesh format:** MFEM reads Gmsh v2.2 only (`miniapps/seas/CLAUDE.md`); the `-format msh22` flag is mandatory in the regen step.

Acceptance Criteria

- `tpv31/configs/tpv31_rk_mixedflux.toml` created (diff vs `tpv31.toml`: `mixed_flux: none→adjacent PLUS [material].seam_continuous = true`); `interior_flux="matrix"` preserved.
- `sbatch` created; `module/LD_LIBRARY_PATH` block byte-identical to the working TPV31/TPV104 jobs; pre-flight greps pass; restart chain present; no `--dt`.
- Test 5.1 + Test 5.3 (parse/dry-run, tiny fixture) confirm `matrix + adjacent + rk4` clears BOTH G1 (`spatial_friction.cpp:1162`) and G2 (`driver:811`); `[material].seam_continuous` parses; and the existing spatial TOMLs (`tpv205/102/104_spatial` + `tpv31 p1/p2/p3`) still parse field-for-field unchanged (BUG-21/BUG-22/BUG-25 proxies before any Frontera run; BP5 excluded — it uses the native driver, not this parser).
- Tests 3.4b/3.4c (`np=2`) pass: a `Mode::Coefficient` central shared face without `seam_continuous=true` aborts; a depth-profile fixture WITH it builds (no false-abort).
- `tpv31/visualize_results.py` gains `--tol-rms/--tol-peak` with interpolation over the explicit `[t0,t1]` overlap window + minimum-coverage guard + peak-normalized, named-channel (exclude `mu_eff`) metric, non-zero exit on exceedance; `benchmark_data/README.md` updated to the real .txt format. Tests NEW-7.1 (offset grids = interpolation; `>band = exit≠0`; within-band = exit 0), NEW-7.4 (zero-crossing = no spurious fail), NEW-7.5 (truncated run = insufficient-coverage `exit≠0`) pass locally.
- **[Frontera, user-approved]** `idev slice` on `tpv31_50m` exits cleanly; a whitespace-tolerant grep of the *.out finds the banner time integrator: `+ mixed flux: lines AND the --verify-dispatch interior flux line` (BUG-13 — not brittle exact strings).
- **[Frontera, user-approved]** the 15 s run advances with BOUNDED `V_max` (no central-flux runaway), NaN-free; `visualize_results.py --tol-rms --tol-peak` vs `scex_seisol` **exits 0** within the documented ~5-10% peak band.
- The existing TPV31 ADER/noflux jobs + TPV*/BP5 byte-exact suites unchanged.

Dependencies

- Depends on: Phases 1-6. Required by: nothing (final deliverable).

Testing Strategy

Level	Test	Guards against
Primitive	<code>test_bimaterial_central_wave</code> (1.1-1.3)	wrong central formula; homogeneous-limit drift; dropping <code>A_nbr</code> (1.3)
Dispatch	<code>test_bimaterial_mixed_flux</code> (3.1-3.3)	wrong dispatch kernel selection; non-single-valued central deposit (3.2, het)
Dispatch (MPI)	<code>test_bimaterial_mixed_flux</code> (3.4a/3.4b/3.4c, <code>np=2</code>)	shared-face keying (BUG-5, 3.4a); <code>seam_continuous</code> gate (BUG-6/BUG-10/BUG-15, 3.4b); no false-abort on depth profile (BUG-16, 3.4c)
CFL	<code>test_bimaterial_mixed_flux</code> (4.1-4.2)	factor divergence vs scalar (BUG-2); central+ADER
Equivalence	<code>test_bimaterial_mixed_flux</code> (6.1)	flux path equivalence vs scalar mixed-flux path (necessary, not sufficient)
Driver	<code>parse/dry-run</code> 5.1-5.3	config-parse guard G1 (BUG-21); driver guard G2 (BUG-4); <code>matrix+ADER</code> must abort; <code>seam_continuous</code> parse + parser-regression (BUG-22)
Tooling (Phase 7)	<code>test_visualize_results</code> (NEW-7.1/7.4/7.5)	missing gate (BUG-11); no interpolation (BUG-19); undefined denominator (BUG-17); unnamed channel (BUG-18); overlap window + truncated-run coverage (BUG-23); stale README (BUG-12)
Parity (existing)	<code>test_bimaterial_wave_operation</code> (22/22)	regression parity <code>mixed_flux=none</code>
Regression (existing)	TPV102/104/205 + BP5 + TPV31-ADER byte-exact	any leak into production numerics
Integration (Frontera)	TPV31 RK45 + adjacent (Phase 7)	end-to-end matrix+central+RK; SeisSol overlay (gated by NEW-7.1's exit code)

Validate correctness against: (a) the Phase-1 analytic/homogeneous reductions, (b) the existing scalar mixed-flux operator as the reference for the homogeneous limit, (c) drdg3d's `get_flux` structure as the design oracle, (d) **the SCEC SeisSol TPV31 reference on Frontera** for the quantitative bi-material rupture (transmission/reflection at the depth contrasts).

Risk Assessment

- **Central-flux single-valuedness (BUG-3, highest risk).** Depositing $\frac{1}{2}A_{\text{self}} \cdot (Q_{\text{self}} + Q_{\text{nbr}})$ per side, or reusing the upwind per-side operand swap for the central build, breaks conservation under heterogeneity and still passes homogeneous tests. *Mitigate:* the Phase-2 **side-symmetric store** (one pair per face, no swap) + Phase-3 **F_h_e2 ← F_h_e1 copy**. *Detect:* **Test 3.2 on heterogeneous material** (operator-level $F_{h_e1} == F_{h_e2}$) — NOT the Phase-1 primitive, which only checks commutativity (BUG-7).
- **Residual driver guard (BUG-4).** `spatial_dyn_driver.cpp:809-816` rejects `matrix + RK`, which mixed flux requires; the feature cannot run end-to-end until it is relaxed (Phase 5). *Detect:* Test 5.1 + the Frontera dispatch gate (Phase 7).
- **CFL refactor not byte-exact for the bimaterial operator (BUG-2).** The local switch at `bimaterial_wave_operator.inl:521-531` diverges from scalar (`factors + no_cfl_rk_aware_`). *Mitigate:* delete it; route both through `MixedFluxCflFactor_()`; Test 4.1 asserts equality.
- **Parallel shared-face correctness (BUG-5/BUG-6/BUG-10/BUG-15/BUG-16).** Neighbour material is keyed by `sf` not `mesh_face`, and the cross-rank exchange is a local-side stub that stores local as the neighbour (`inl:162`). *Mitigate:* `sf`-loop shared arm (Phase 2 req 3); restrict the merged feature to Constant/seam-continuous material (TPV31 qualifies). **Enforce the boundary with the declarative material.seam_continuous flag (Phase 2 req 4) — NOT a stored-neighbour-vs-local compare (dead, always-equal, BUG-10) and NOT a material-field probe (no arbitrary-point evaluator on material_, BUG-15; false-aborts on legitimate depth variation, BUG-16).** *Detect:* Test 3.4a (build), Test 3.4b (missing-flag abort), Test 3.4c (depth-profile no-false-abort). The fully-correct cross-rank `MPI_Allgather_v` exchange is deferred.
- **Orientation/frame mismatch** between central and Godunov per-face matrices. *Mitigate:* the central builder reuses the Godunov `BuildFrame/BuildRotation` helpers (Phase 1 req 2); identical (`flux_e1, flux_e2, nor`) operands.
- **Refactor breaking byte-exactness** (ctor operand factoring). *Detect:* parity test (22/22) must stay green; do the refactor as a no-op commit first. The CFL extraction is byte-exact for the SCALAR operator only (see BUG-2).
- **ADER + central runaway.** *Mitigate:* Phase 4 + Phase 5 hard guards abort central+ADER on BOTH the operator and the driver (consistent condition + message).
- **CFL factors are empirical placeholders** (`wave_operator.inl §14.4`). The merged path inherits the scalar factors; final stability is a Frontera calibration (Phase 7), not a local pass/fail. Flag in the benchmark doc; do not hardcode a “validated” claim. *Honors* the explicit-RK CFL memory (omit `--dt`, use auto-CFL).
- **Frontera run policy.** Phase 7 stages the job; the user submits after explicit approval (project memory). The plan never auto-submits, `ssh`'s, or checks the allocation.

Tricky existing-code areas this plan touches

- `bimaterial_wave_operator.inl:567-576` — the **actual** `SetMixedFluxMode` abort to replace (BUG-1), NOT `wave_operator.inl:1623-1626` (a comment).
- `bimaterial_wave_operator.inl:269-324` ctor build lambdas — refactor into `ResolveFaceFluxOperands_` with care; the per-side swap is for UPWIND only (BUG-3).
- `bimaterial_wave_operator.inl:375-422` upwind shared-face Pass 2 — the structure the central shared arm mirrors (`sf`-keyed; BUG-5).
- `bimaterial_wave_operator.inl:120-190` `ExchangeBiMaterialNeighbours_` — local-side stub (stores local-as-neighbour, `inl:162`); the BUG-6/BUG-10/BUG-15/BUG-16 support boundary. Enforce it with the declarative `material.seam_continuous` flag (Phase 2 req 4) — NOT a stored-neighbour-vs-local compare (dead) and NOT a material-field probe (`MaterialField` has no arbitrary-point evaluator the operator can reach, `heterogeneous_material.hpp:183-209`; an along-normal probe false-aborts on depth variation).
- `bimaterial_wave_operator.inl:515,521-531,539-555` `ComputeMaxDt` — delete the local CFL switch (BUG-2); keep the per-element walk + `Allreduce`.
- `wave_operator.inl:1650+` `BuildCentralFluxFaceSet_` — reuse verbatim (R-1100/R-1206/R-1404); do not duplicate.
- `wave_operator.inl:5880-5903` `ComputeMaxDt` switch — single source of truth after extraction.

- **Series-guard inventory (relax ALL THREE — Phase 5):** `spatial/code/spatial_friction.cpp:1162-1167` (G1, `config-parse`, fires FIRST — BUG-21), `spatial_dyn_driver.cpp:809-816` (G2, driver matrix+RK — BUG-4; 806-808 stale comment), `bimaterial_wave_operator.inl:569-574` (G3, operator — BUG-1). KEEP (TPV31 does not trip): `spatial_friction.cpp:1702-1707` (matrix needs non-Constant), `spatial_dyn_driver.cpp:1078-1085` (scalar needs Constant), `wave_operator.inl:604-610/1613-1616` (precomputed-flux $\leftrightarrow$ mixed mutual exclusion). Driver wiring seams: 1099-1138/1245/1902-1905.
- `spatial/code/spatial_friction.{hpp,cpp}` — `MaterialSpec` (hpp:573) + `[material]` parser (cpp:1626+): add `seam_continuous` (BUG-22). Every TPV/TPV31 `spatial*` config parses here (NOT BP5 — native driver) = parser-regression test required (Test 5.3c, via `test_tpv_config_parse.cpp`, field-by-field). Scope-expanded in Rev 5 (Constraints).
- Stale comments to update: `bimaterial_wave_operator.inl:518-520`, `.hpp:121`, `wave_operator.inl:1623-1626`, `spatial_dyn_driver.cpp:806-808` (BUG-1).
- `makefile:311-317`, 859-893, 1638, 5248, 5252 — the 4-edit-per-test registration pattern (BUG-8).
- `tpv31/visualize_results.py` (`argparse` ~373-437; reader 158-192; ref glob 201,575) — add the `--tol-rms/--tol-peak` per-station RMS+peak pass/fail gate (BUG-11). Benchmark tool, in-scope.
- `tpv31/benchmark_data/README.md` — stale (claims dir empty + `.dat`; reality 60 `.txt`); rewrite to match the committed data + the script's parser contract (BUG-12).